

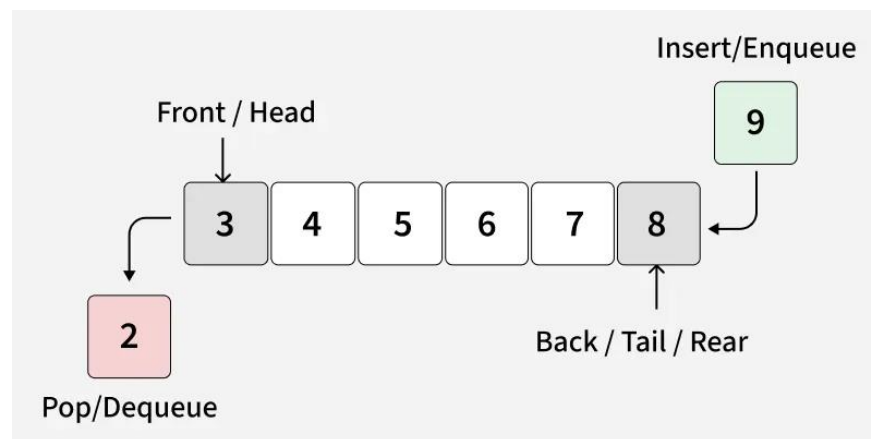
Linear - Dynamic Data Structure (Queue)

Introduction

A **queue** is a **linear** data structure that consists of elements arranged in a **sequential order** where **one end is used to add** elements and **another for removing** them. In this lecture, we will learn how to implement queues using an array.

Queue Properties

1. It works on **FIFO** (**F**irst **I**n **F**irst **O**ut) principle of states that the first element added to the Queue will be the first one to be removed or processed. So, Queue is like a line of people waiting to purchase tickets,
2. It has mainly (2) pointers to perform operations: **Front** & **Rear**. The **Rear** is for **insertion** (called an enqueue operation) and **front** for **removing** (called a dequeue operation).
3. It is a **linear data structure** that gives **sequential one-way** access to the elements.
4. You cannot access a queue's elements by index directly because access to elements is given only through the **rear** and **front pointers**.



The figure above shows an array with various cells:

1. Initially our **array based Queue** consists of : [2, 3, 4, 5, 6, 7, 8]
2. And then we perform an **Enqueue** and a **Dequeue** Operation for element 2 and 9 respectively.
3. After performing operations, our final queue is : [3, 4, 5, 6, 7, 8, 9].

Basic Terminologies of Queue

1. **Front:** Position of the entry in a **queue** ready to be served, that is, the first entry that will be removed from the queue, is called the **front** of the queue. It is also referred as the **head** of the queue.
2. **Rear:** Position of the last entry in the **queue**, that is, the one most recently added, is called the **rear** of the **queue**. It is also referred as the **tail** of the **queue**.
3. **Size:** Size refers to the **current** number of elements in the **queue**.
4. **Capacity:** Capacity refers to the **maximum** number of elements the queue can hold.

Basic Operations on Queue

In order to implement a **queue using array**, we must be aware of the basic operations that are performed in a queue to manipulate the elements present inside it. Following are the basic operations of the queue data structure:

Sq.	Operation	Description
1	Enqueue	Inserts an element into the queue using the rear pointer .
2	Dequeue	Deletes an element from the queue using the front pointer and returns it.
3	Size	Returns the total numbers of elements present in the queue
4	front()	Returns the element at the front end without removing it.
5	rear()	Returns the element at the rear end without removing it.
6	isEmpty	Returns true if the queue is empty .
7	isFull	Returns true if the queue is fully and there is no space left .

The two main operations related to the queue data structure are **enqueue** and **dequeue**, Let us understand about the implementation of them.

1. Enqueue Operation:

Enqueue operation **adds (or stores) an element to the end of the queue**. The following steps are the algorithm to perform the **Enqueue** operation:

Algorithm of Enqueue

- Step 1:** Check if the **queue is full** using the **isFull** function.
- Step 2:** If the queue is full, return an **overflow** error and exit.

Step 3: If the queue is **not full**, **increment** the **rear** pointer to the next empty space.

Step 4: Insert (add) the new element to the **rear** position.

Step 5: return success.

2. Dequeue Operation:

Dequeue operation removes the element at the front of the queue. The following steps are taken to perform the dequeue operation:

Algorithm of Dequeue

Step 1: Check if the **queue is empty** using the *isEmpty* function.

Step 2: If the queue is empty, return the **underflow** error and exit.

Step 3: If the queue is **not empty**, **remove** the element at the **front** position.

Step 4: **Increment** the **front** pointer to point to the next available data element. Step 5: The Return success.

3. Front Operation

This operation returns the element at the **front** of the queue. It is the **elements** that will be **removed** first when we perform dequeue. The following steps are taken to perform the **Front** operation:

Algorithm of Front

Following is the algorithm for the front operation:

Step 1: Check if the queue is **empty**. If the queue is **empty**, **return -1**.

Step 2: If the queue is **not empty**, return the element pointed by the **front** pointer.

4. IsEmpty Operation

This operation checks whether the given queue has any element or is it empty. This function returns **true** if the queue is **empty**, otherwise returns **false**. We know that when the queue is **empty** initially, the **front = -1**.

Algorithm of isEmpty

Following is the algorithm for the isEmpty operation:

Step 1: If **front = -1** OR **front > rear** is **true**, then we can say that the queue is **empty**.

Step 2: It evaluates as **false**, and then we can say that there are elements in the queue.

5. IsFull Operation

The **isFull** operation checks if the queue is **full**. It means that whether it contains **n** elements where **n** is the **size** of the array using which the queue is implemented.

Algorithm of isFull

Following is the algorithm for the isFull operation:

Step 1: Check if the **rear = max_array_size - 1**.

Step 2: If it is **true**, then the queue is **full**.

Step 3: If its **false**, then it means that there is **space** left in the queue.

Home work:

Write a python program that includes an interface design containing multiple choices for the user to implement the following operations

1. Insert the elements; (4, 3,6,3,8 and 5),
2. Dequeue one element,
3. Display the queue elements
4. Dequeue three element,
5. Display the queue elements
6. Lastly to exit the program